

Performance Analysis of Dense and Sparse Matrix Multiplication

Individual Assignment Report

Anna Sowińska DEY61301

Course: **Big Data 40955**

GitHub Repository: https://github.com/asowinska9/matrix_project

November 21, 2025

Abstract

This report presents a comparative performance evaluation of matrix multiplication implemented in Python, C++ and Java. Both dense (naive matrix–matrix multiplication) and sparse computations (matrix–vector and matrix–matrix operations using the CSR format) were benchmarked on synthetic data and on the large real-world sparse matrix `mc2depi.mtx`. The study follows the assignment guidelines: separation of production and test code, parametrised experiments, several runs per configuration, and basic profiling. The results show that compiled languages (C++ and Java) are significantly faster than pure Python loops and that exploiting sparsity is crucial for performance on large sparse problems.

1 Introduction

Matrix multiplication is a fundamental operation in numerical linear algebra, scientific computing and machine learning. The objective of this work is to evaluate the performance of dense and sparse matrix multiplication implemented in Python, C++ and Java.

2 Methodology

2.1 Implementations

We implemented:

- **Dense naive** multiplication using a standard three-loop structure.
- **Sparse CSR** operations (SpMV and SpMM).

Python uses only the standard library, while C++ and Java are compiled/executed with optimisations.

2.2 Experimental Setup

Dense tests use sizes $n = 64\text{--}512$, sparse tests use the `mc2depi` matrix. Each experiment was repeated 5 times.

2.3 Performance Metrics

FLOP counts follow the standard formulas for dense and sparse kernels.

3 Dense Results

3.1 Execution Times

Table 1 summarises median execution times.

Table 1: Dense matrix–matrix multiplication (naive).

Language	n	Time [s]
Python	128	0.264
Python	256	1.852
Python	512	14.571
C++	64	0.00062
C++	128	0.00543
C++	192	0.02217
Java	64	0.00098
Java	128	0.00373
Java	192	0.01818

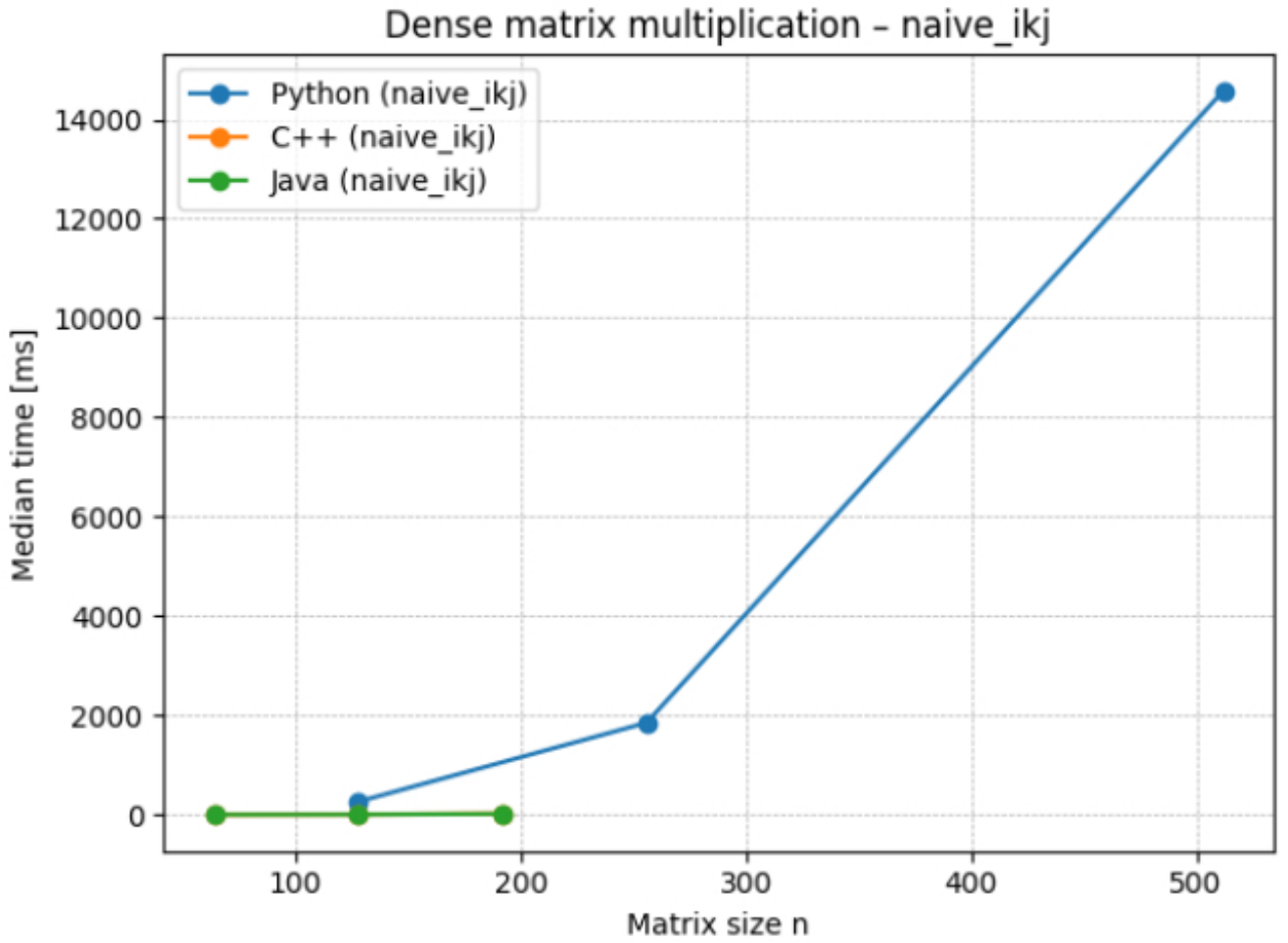


Figure 1: Execution time for dense matrix multiplication in Python, C++ and Java.

4 Sparse Results

4.1 Dataset: mc2depi

The matrix has $n = 525,825$ and $nnz = 2,100,225$.

4.2 SpMV Results

Table 2 and Figure 2 present the performance.

Table 2: SpMV on mc2depi. Median of 5 runs.

Language	Median time [ms]	GFLOP/s	Speedup
Python	767.988	0.0055	1×
C++	6.370	0.66	$\approx 120\times$
Java	10.618	0.40	$\approx 72\times$

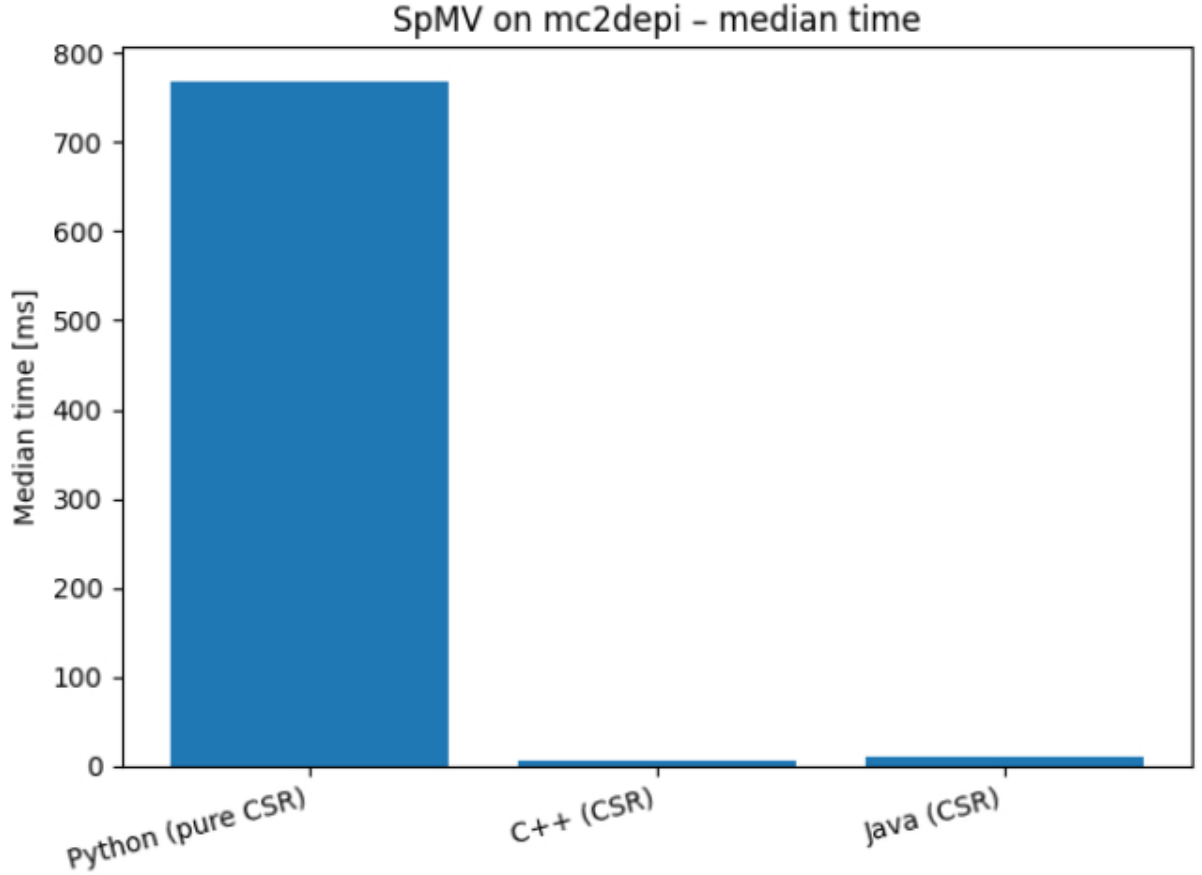


Figure 2: SpMV performance comparison across languages.

4.3 SpMM ($k = 16$)

Table 3 and Figure 3 summarise SpMM results.

Table 3: SpMM ($k = 16$) on mc2depi. Median of 5 runs.

Language	Time [ms]	GFLOP/s	Speedup
Python	5963.726	0.011	$1\times$
C++	57.368	1.17	$\approx 104\times$
Java	113.746	0.59	$\approx 52\times$

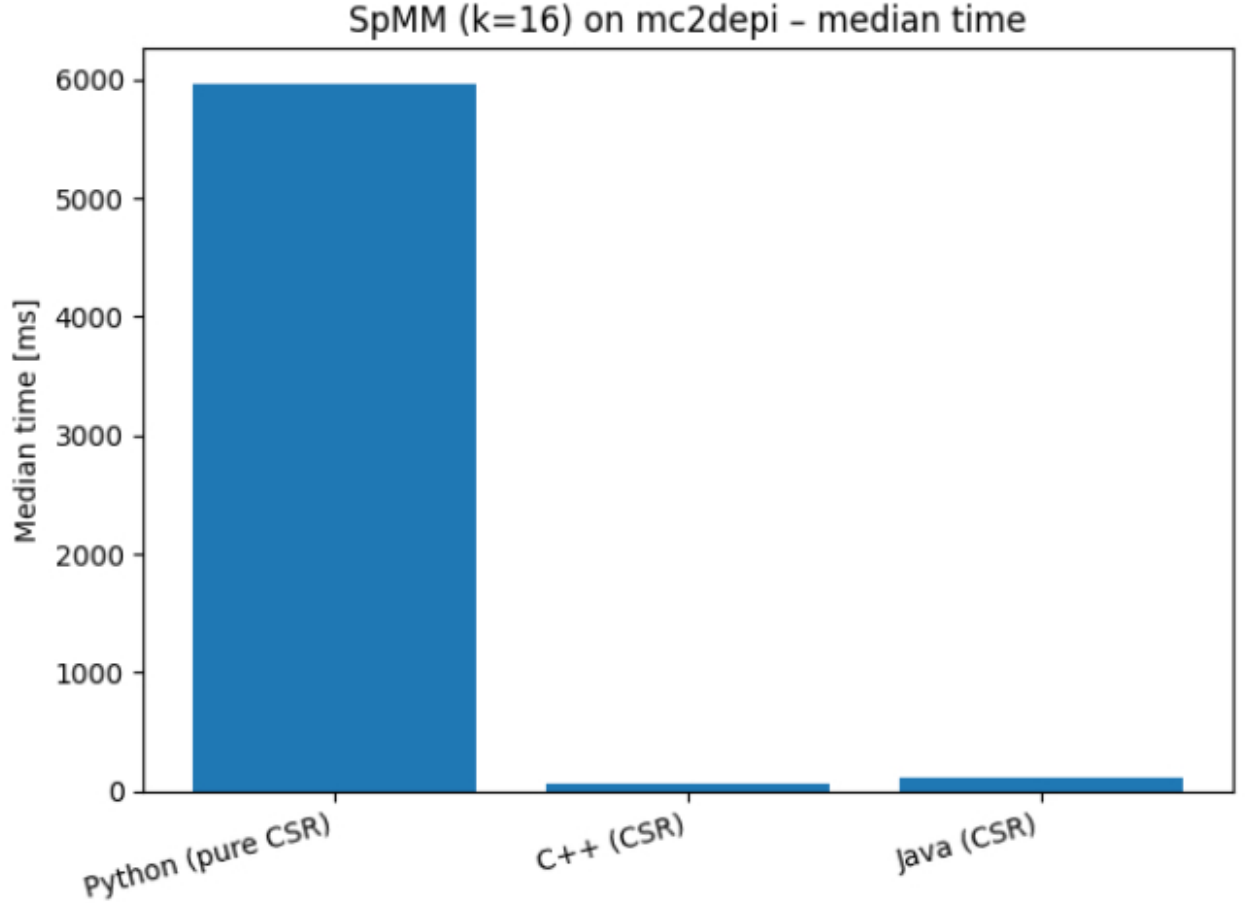


Figure 3: Execution time of SpMM for Python, C++ and Java.

4.4 Scaling with k in C++

Figure 4 shows scaling with increasing dense columns.

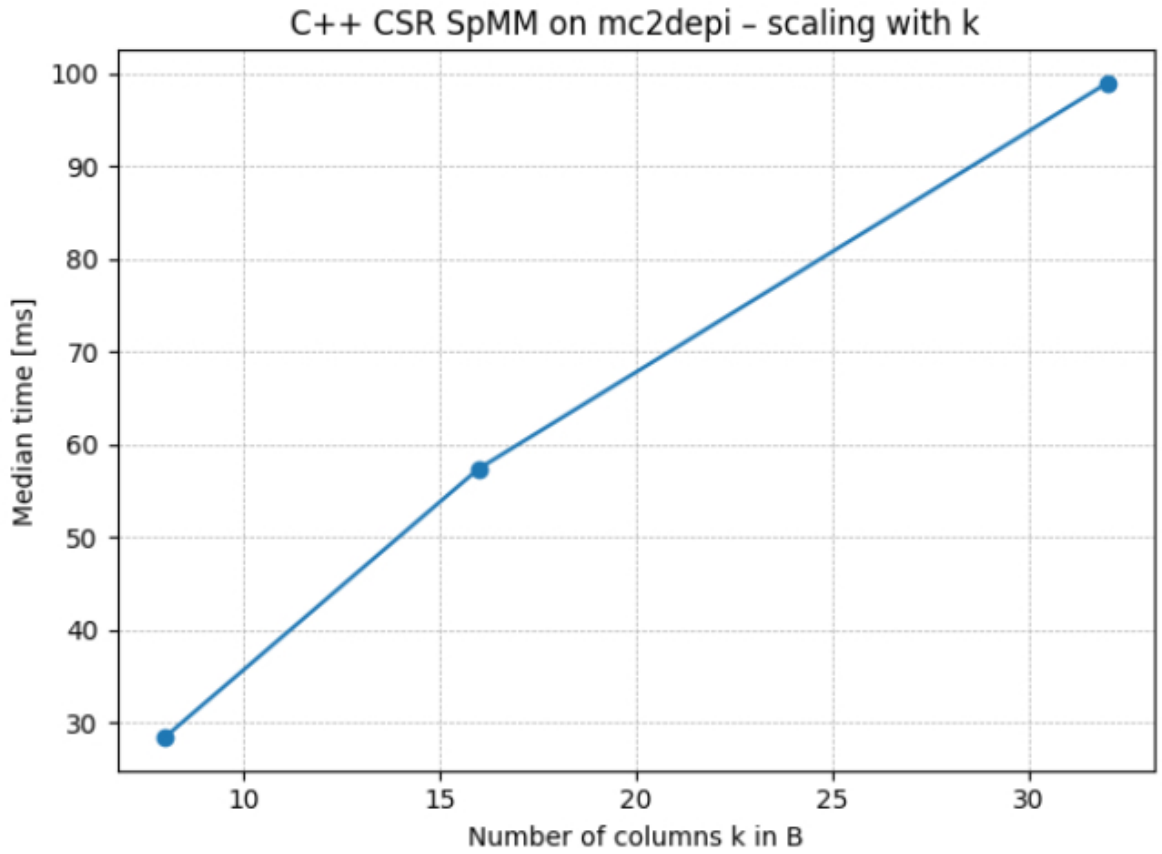


Figure 4: C++ SpMM scaling with number of columns k .

5 Discussion

Compiled languages outperform Python significantly. Sparse formats enable feasible computation on extremely large matrices.

6 Conclusions

Key conclusions:

- C++ achieves the highest performance.
- Python pure loops are unsuitable for heavy numerical kernels.
- CSR is essential for large sparse matrices.
- SpMM scales linearly with k as predicted.

Acknowledgement

Parts of this report were prepared with assistance from AI-based tools.

References

- [1] S. Williams et al., “Optimization of Sparse Matrix-Vector Multiplication on Emerging Multicore Platforms”, *Parallel Computing*, 2009.
- [2] T. A. Davis and Y. Hu, “The University of Florida Sparse Matrix Collection”, 2011.